Architecture & Design Guide

AstroIllustrator | Consolidated specification v2.1 | 22 October 2026

1. Architectural choice

Use a lightweight Clean Architecture-inspired modular monolith. Api is transport/composition, Core owns types/validation/ports, Infrastructure owns EF Core, local storage, AI provider adapters and Python invocation. Python owns scientific computation and vector plotting. Do not add a separate Application project until orchestration complexity justifies it. No microservices, CQRS/MediatR, distributed queues or generic repository wrappers in MVP.

2. Projects and dependencies

AstroIllustrator.Api -> Core + Infrastructure; Infrastructure -> Core; Core -> no project references. renderer/ is a separate Python package called by a process adapter with a stable JSON protocol. Optional AstroIllustrator.Tests project is worthwhile early for domain validation, API contracts and integration tests.

3. Core domain abstractions

IllustrationRequest discriminates SkyChartRequest, ScientificDiagramRequest and ArtisticCompositeRequest. IIllustrationValidator validates the common envelope; ISceneRequestValidator and IDiagramRequestValidator validate type-specific constraints. IRenderService orchestrates render jobs; IRendererAdapter abstracts Python execution; IOutputStore persists files; IRenderJobStore handles job metadata. Keep interfaces only at meaningful external boundaries; avoid an interface for every class.

4. Renderer internals

In Python, keep a scene dispatcher selecting sky-chart, diagram or composite rendering. Use a Strategy-like handler registry by scene/diagram type, not a sprawling conditional tree. Share geometry primitives and export pipeline. Separate CatalogRepository (read-only scientific files) from EF Core metadata persistence. CoordinateTransforms, ProjectionService and DiagramGeometry should be deterministic, testable modules.

5. Patterns to use selectively

Adapter: Python CLI and optional LLM/image-provider APIs. Strategy: rendering mode and projection choice. Factory: validated handler selection. Options pattern: catalog paths, timeouts, output profiles. Dependency injection: constructor-based. Avoid generic repository and Unit of Work on top of EF Core DbContext unless a specific testability or persistence boundary requires it. Avoid a rules engine for a small number of diagram templates.

6. AI integration boundary

IIllustrationPromptInterpreter converts a prompt to typed configuration; a provider-specific adapter implements it. Validation occurs after the model returns JSON and before any renderer action. IBackgroundGenerator is optional and must never touch chart geometry. If the user requests exact Polaris altitude without time, request missing observation time. If schematic mode is selected, mark the result approximate and explain the NCP-latitude relationship.

7. Errors, testing and observability

Use ProblemDetails in API; typed validation errors; Python subprocess exit code, stderr and timeout handling; cancellation; bounded concurrency; safe paths; logs with renderJobId. Unit tests for geometry/angles, catalog ID lookup and request validation; integration tests for JSON bridge; reference tests for Orion/Summer Triangle/horizon/Polaris; image snapshots for typography only, numeric geometry tests for scientific truth.

8. Future-proofing without extra layers

Version the JSON request schema, diagram templates, catalog snapshots and rendering engine. Store a manifest per figure. New diagrams can be added as a template and handler without changing project topology. Use SQLite locally. Switch to PostgreSQL only if multi-user/concurrent server deployment makes it necessary. Document control: This

version supersedes earlier 1.0 and 1.1 scope descriptions. Keep requests, manifests, diagrams and AI integrations aligned with this specification.

v2.1 additions: global observer support, time-aware conjunctions and AstroIllustrator naming

Renamed Visual Studio solution

AstroIllustrator.sln; src/AstroIllustrator.Api, src/AstroIllustrator.Core, src/AstroIllustrator.Infrastructure; tests/AstroIllustrator.Core.Tests and AstroIllustrator.Api.Tests; renderer/astro_renderer/{astronomy,ephemeris,scenes,diagrams,exports}; docs/. References: Api -> Core and Infrastructure; Infrastructure -> Core; Python renderer is process-isolated via typed JSON.

Focused services

Core contracts: IObserverLocationValidator, ITimeReferenceResolver, IAstronomicalEventService, IRenderJobService. Infrastructure: PythonRendererClient, EphemerisAdapter and EF metadata stores. Python modules: observer.py, solar_events.py, planets.py, moon.py, event_search.py, horizon_event_scene.py. Keep calculations in Python and orchestration/validation in .NET.

Boundary invariants

An LLM never directly supplies trusted ephemeris coordinates, conjunction dates or measured angles. The AI adapter returns a candidate request; Core validates it; scientific engine computes facts; renderer draws. Store ephemeris version/hash, observer, UTC instant, time-scale assumptions, refraction settings and projection in manifest.

No overengineering

No fourth .NET project, broker, CQRS, or database-driven star catalog required for event support. SQLite remains for job metadata and presets; astronomical kernels are versioned file assets. Add service interfaces only when they clarify testability or provider boundaries.

Scientific note: illustrative dates and prompts are examples, not verified event predictions. All published charts require computed ephemerides and editorial review.